VecPAC: A Vectorizable and Precision-Aware CGRA

Cheng Tan[†], Deepak Patil[§], Antonino Tumeo[*], Gabriel Weisz[¶], Steve Reinhardt[¶], Jeff Zhang[§]
[†]Google, [¶]Microsoft, [*]Pacific Northwest National Laboratory, [§]Arizona State University
[†]chengtan@google.com, [¶]{gabriel.weisz, stever}@microsoft.com, [*]antonino.tumeo@pnnl.gov,
[§]{dppatil3, jeffzhang}@asu.edu

2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD) | 979-8-3503-2225-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCAD57390.2023.10323910

***Abstract*—Coarse-grained reconfigurable arrays (CGRAs) are a promising solution to accelerate applications from several domains, thanks to their balance between the performance achieved through specialization and the adaptability to obtain different computational patterns through dynamic reconfiguration. Several state-of-the-art CGRA designs try to further exploit domain specialization by integrating additional specialized functional units to support custom numeric formats and/or vector functional units. While this approach can improve performance and efficiency for kernels coming from a single application domain, it lowers the overall utilization of the hardware resources and reduces the adaptability of the accelerator.**

This paper proposes VecPAC – a vectorizable and precision-aware coarse-grained reconfigurable array (CGRA) design. VecPAC integrates CGRA tiles with scalar functional units and specialized tiles with vector functional units that can trade off the number of vector lanes for the accuracy of the computation. We discuss the architecture design and present the related compilation framework. The experimental evaluation on a set of applications from three different domains (embedded, machine learning, and high-performance computing) shows that the hybrid design of VecPAC outperforms CGRAs with only scalar functional units by 1.48×, while providing higher scalability (evaluated on 2×2, 4×4, and 8×8). Moreover, VecPAC achieves better area-efficiency (1.74×) over a CGRA with only vector functional units.

I. Introduction

Coarse-grained reconfigurable arrays (CGRAs), loosely defined as tiled architectures composed of functional units (FUs) interconnected through an on-chip network, promise higher efficiency than fine-grained reconfigurable devices (such as field-programmable gate arrays - FPGAs), while maintaining adaptability to different computational patterns [16]. CGRAs have applications in embedded [13], machine learning (ML) [14, 29], and high-performance computing (HPC) [10] domains.

However, different domains exhibit different computational characteristics. For example, HPC and ML (on fully interconnected neural networks) algorithms have high data parallelism that benefits from single-instruction, multiple-data (SIMD) units. On the other hand, image and signal processing, and many ML models can afford some loss of accuracy when using numeric formats with lower precision. Both these optimizations typically result in higher power and area efficiency for designs targeted to specific domains. Recently, applications that employ mixed approaches that include scientific simulation, data analytics, and ML have appeared, needing accelerators that could support equally well algorithms with different requirements and behaviors.

Current design approaches for CGRAs either focus on specializing the hardware for a specific domain [24, 27, 29] or on providing additional FUs supporting vectorization and/or custom numeric formats in addition to existing resources [2, 20]. The first approach does not efficiently map applications from other domains, while the second is not energy and area efficient, resulting in resource under-utilization when mapping applications from domains that have different accuracy requirements. One opportunity to increase efficiency is to support reconfigurability at the level of the vector units. It is, in fact, possible to trade-off the number of vector lanes (and thus data parallelism) with precision depending on the application requirements. To support such an architecture it is necessary to address several issues at the hardware and software level, namely the design of the configurable vector FUs and the mapping algorithms.

This paper proposes VecPAC, a Vectorizable and Precision-Aware CGRA design and its software infrastructure. Its contributions are:

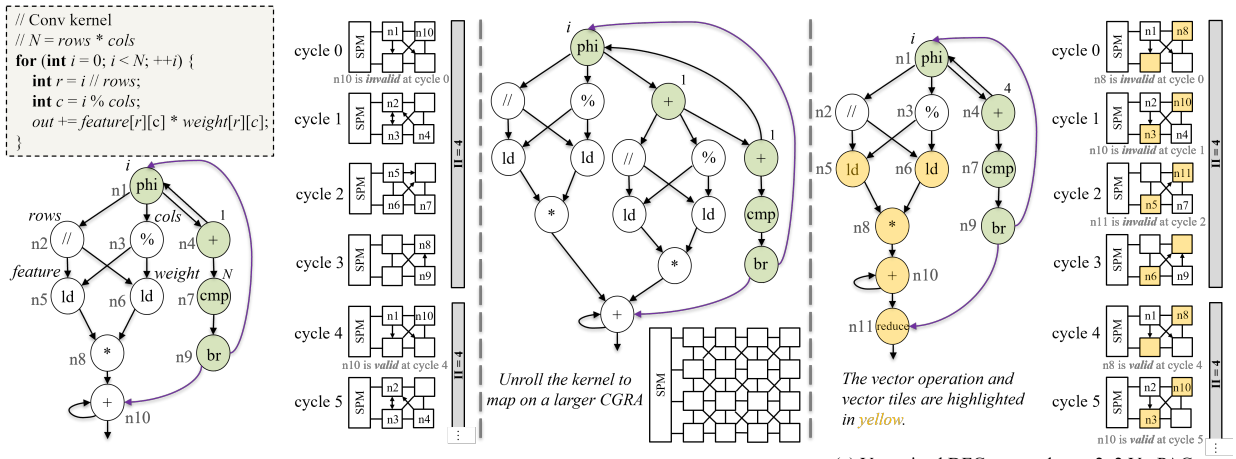
- a novel hybrid CGRA architecture (VecPAC) that contains both tiles with scalar or vector FUs to efficiently accelerate vectorizable application kernels;
- a vector FU design that can trade-off number of lanes with accuracy: in the proposed design, vector lanes can be fused together to support more accurate number formats;
- the software stack to map vectorizable applications;
- power, performance, and area evaluation of the RTL design on a set of representative domain-specific application kernels; and a backend ASIC flow that implements the CGRA with ASAP 7nm node.

II. Related Work

Table I compares VecPAC with other CGRA designs that support vectorization or are precision-aware.

Vector CGRAs – Plasticine [21] is composed of reconfigurable Pattern Compute Units (PCUs) and Pattern Memory Units (PMUs) to accelerate parallel patterns. The PCUs consist of multiple pipelined compute SIMD lanes that perform vector computations. Similarly, HEAL-WEAR [8] duplicates the datapath in each reconfiguration cell (i.e., tile/PE) to enable vector computations targeting electrocardiogram (ECG) applications. Riken [20] is a CGRA template used to perform design space exploration of CGRAs for HPC applications. Each tile in Riken can be configured as a SIMD engine with vector lanes.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.



(a) Conv kernel, DFG, and a mapping on a 2x2 CGRA. (b) The DFG of the unrolled Conv kernel. (c) Vectorized DFG mapped on a 2x2 VecPAC.

Fig. 1: Motivating example – Control-flow is converted into data-flow using partial predication. The DFGs are simplified for clarity (e.g., flattening the nested-loop, eliminating the phi node around the accumulation, and removing the address calculation node before the `ld` operation). The critical path is highlighted in green in the DFG.

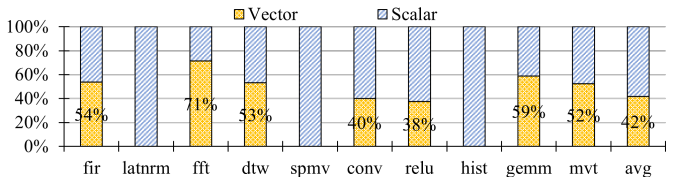


Fig. 2: Proportion of vectorized and scalar operations across a set of widely used kernels – We compile the kernels using LLVM with Auto-Vectorization.

Unfortunately, all previous works focus on accelerating applications with high data parallelism and implementing vector functional units for all the tiles/PEs across the CGRA fabric. This leads to low performance and/or area efficiency when considering applications with kernels difficult to vectorize. None of these solutions consider precision and specialized numeric formats as a design point, thus not evaluating accuracy implications.

CGRA Design	Vector Support	Hybrid Design	Precis. Aware	Fusible Precis.
Plasticine [21]	✓	✗	✗	✗
HEAL-WEAR [8]	✓	✗	✗	✗
Riken [20]	✓	✓	✗	✗
TRANSPIRE [22]	✓	✗	✓	✗
PX-CGRA [4]	✗	✗	✓	✓
BitFusion [23]	✓	✗	✓	✓
ASAP [25]	✗	✓	✓	✗
BioCare [9]	✓	✗	✓	✓
VecPAC	✓	✓	✓	✓

TABLE I: The state-of-the-art CGRAs with vectorization support and/or precision-awareness – Precis. Aware and Fusible Precis. indicate precision-aware and solutions where functional units can be fused to increase the precision of the computation, respectively.

Precision-Aware CGRAs – TRANSPIRE [22] leverages transprecision floating-point units [17] that can operate on several floating-point number formats in a SIMD fashion. Similarly, PX-CGRA [4] employs polymorphic-approximated ALU clusters to model an accuracy-configurable CGRA. However, both designs integrate additional arithmetic units, which significantly increase their areas. ASAP [25] is a hardware/software co-design framework that performs automatic synthesis of area-efficient and precision-aware CGRAs for a given set of applications of interest. Whereas, it doesn't support vector computations. VecPAC's ability to fuse functional units is inspired by BitFusion [23] – a bit-level dynamically composable spatial accelerator for Deep Neural Networks (DNNs). It dynamically matches bit-level composable processing engines to the varying bitwidths required by DNN layers. This approach minimizes the amount of computation and communication required by a DNN at the bit granularity on a per-layer basis. BioCare [9] integrates arithmetic-logic units (ALU) with two different execution modes (8-bit or 16-bit) that allow trading-off accuracy for energy efficiency. However, both BitFusion and BioCare are specialized to only support a single application domain. In contrast, VecPAC is a hybrid CGRA design composed of tiles with scalar functional units and tiles with configurable vector functional units that maximize performance and efficiency across different application domains.

III. Background and Motivation

CGRAs allow accelerating an application kernel by spatially and temporally mapping the operations of its dataflow graph (DFG) onto intercommunicating tiles composed of one or more FUs, mapping communication across tiles according to the data dependencies. The mapping process exploits instruction-level parallelism (ILP) and loop pipelining. Figure 1(a) shows how a `convolution` kernel is mapped onto a 2×2 scalar CGRA. The loop has an initiation interval (II) of 4, meaning that it takes 4 cycles to start a new loop iteration. This provides a speedup of 2.5× (i.e., #nodes ÷ II = 10 ÷ 4) in terms of execution cycles over a single-issue in-order CPU.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

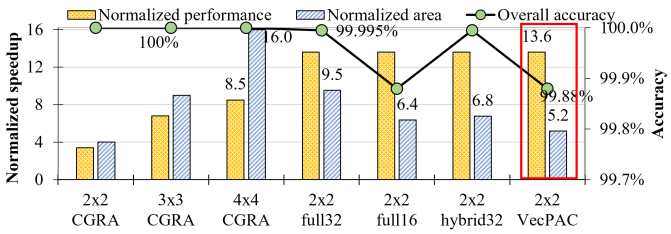
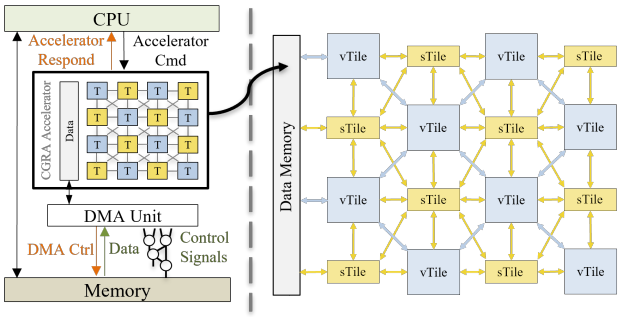


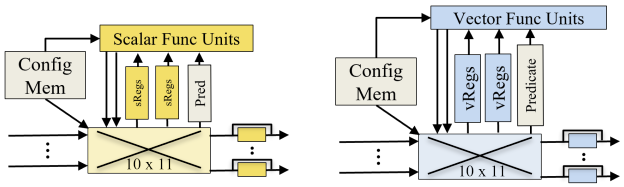
Fig. 3: Normalized speedup, area, and accuracy of different CGRAs accelerating `conv` kernel – The normalized speedup of different CGRAs is the execution cycle improvement over the single-issue in-order CPU. The area is normalized with respect to a single tile. we calculate accuracy as the deviation of the output values from the values obtained by performing all computations of the kernel on a CPU using the IEEE754 floating-point 32-bit format.

Incorporating more computing resources into the CGRA fabric allows to achieve higher speedups, but leads to an area increase. However, to exploit the additional resources, kernels need to be proportionally larger. As shown in Figure 1(b), we can unroll the kernel to increase the size of the DFG and provide more ILP when mapping it to a 4×4 design, leading to higher speedups (i.e., #nodes ÷ (II ÷ unrolling factor) = 10 ÷ (5 ÷ 2) = 4). Increasing the size of the DFG also increases the complexity of the mapping and the probability of having a longer critical path. For example, in Figure 1(b) we can only achieve an II = 5, limiting the scalability of the CGRA. Usually, a kernel can be unrolled using different unrolling factors and pick the one leading to the best performance.

Vectorization can extract data-level parallelism from loops, providing speedup by mapping vectorized operations onto SIMD units. State-of-the-art CGRAs[2, 9, 21, 23] implement vector FUs that improve power- and area-efficiency. Supporting vectorization in CGRAs provides two key benefits. First, a single vector FU with N lanes requires less area than the N parallel scalar FUs required to compute the results of vector operations of size N. Figure 3 shows that a 2×2 CGRA with a 4-lane (each 32-bit fixed-point) vector unit per tile (**full32**) is much smaller than the equivalent 4×4 CGRA with only scalar FUs. The CGRA with vector FUs has fewer tiles, thus fewer configuration memories and crossbars, than the one with scalar FUs, but can perform the same number of operations in parallel, if they are of the same type for each vector FUs. Second, CGRAs with vector FUs do not require aggressive unrolling to extract more ILP to improve mapping opportunities. A single vector instruction performs a number of operations equivalent to the vector length, reducing the mapping complexity and critical path length of the DFG. In Figure 1(c), the vectorized DFG (with a vectorization factor of 2) has only one node more (i.e., the vector reduction operation) than the original DFG (Figure 1(a)). An appropriate mapping can result in a speedup of 5× (#nodes ÷ (II ÷ vectorization factor) = 10 ÷ (4 ÷ 2) = 5) with respect to the equivalent (same number of operations) scalar CGRA. Unrolling and vectorization can be applied together if there are more computational resources available.



(a) The integrated system. (b) VecPAC CGRA architecture overview.



(c) Scalar and vector tile architecture.

Fig. 4: VecPAC architecture – (a) The VecPAC CGRA is loosely coupled with the CPU. (b) The vector and scalar tiles in VecPAC are interconnected through a king-mesh network. (c) The scalar and vector tile architecture.

A. Hybrid SIMD and Scalar CGRA

Figure 2 shows that less than 50% of the operations across a set of representative kernels from different domains are vectorizable. Kernels with loop-carried dependency and unsafe dependent memory operations (i.e., `latnrm`, `spmv`, and `histogram`) cannot be vectorized. Additionally, even in vectorizable kernels, not all operations are vectorizable (e.g., control-flow related operations). Therefore, providing a CGRA with tiles that only have vector functional units cannot guarantee high utilization of the resources.

Figure 1(c) shows that the vectorized operations in the `conv` kernel can be executed by two tiles with vector FUs of the 2×2 CGRA: i.e., it is not necessary to have vector FUs in all CGRA tiles. This motivates the hybrid design of VecPAC. Figure 3 demonstrates that the hybrid designs (the last two 2×2 designs) outperform the others both in performance and area-efficiency. The first three CGRAs are CGRAs with tiles supporting scalar floating-point units at 32-bit, while **full32** and **full16** indicate CGRAs consisting of tiles with fixed-point32 and fixed-point16 vector FUs, respectively. All the vector FUs have four lanes. **Hybrid32** and **VecPAC** indicate hybrid CGRAs containing tiles with both scalar and vector FUs. Scalar tiles consist of floating-point 32-bit FUs, while vector tiles consist of fixed-point32 vector FUs for **hybrid32** and fixed-point16 vector FUs for **VecPAC**.

B. Trade-Off between Accuracy and Performance/Area

Some application domain allows to trade-off accuracy of computation for performance, area, and/or energy by adopting lower precision number formats. For example, ML platforms

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

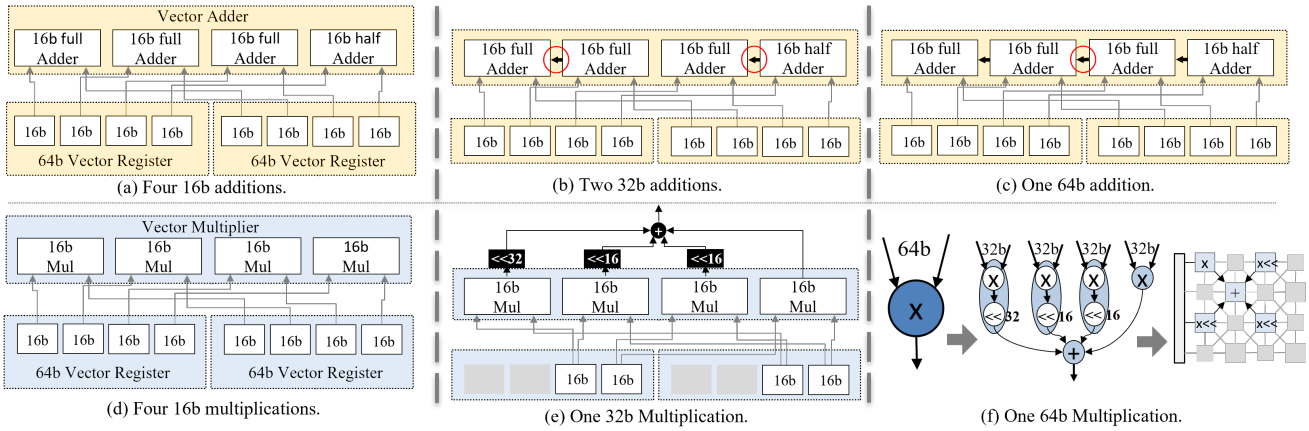


Fig. 5: Vector functional units – (a) Four lanes in the vector adder execute independently to support 4-way vectorization. (b) Each two lanes in the vector adder fused together to support fixed-point32 and form a 2-way vector adder. (c) All four lanes fused together to support fixed-point64 addition as a scalar adder. (d) 4-way vector multiplier. (e) Four lanes fused together to support fixed-point32 multiplication. (f) A 64-bit multiplication in the DFG can be represented as five 32-bit operations and mapped on five vector tiles in the CGRA for acceleration.

(NVIDIA GPUs, Google TPUs, etc.) implement specialized formats, such as TensorFloat32 [5], Bfloat16 [3], and INT4/8, that improve energy efficiency with limited or no accuracy loss (thanks to fine-tuning). CGRAs can also implement FUs with several number formats. However, implementing additional FUs for each format might increase area and reduce resource utilization. As shown in Figure 3, a CGRA with vector FUs at fixed-point16 (**full16**) occupies 67.4% (i.e., 6.4 ÷ 9.5) of the area of a CGRA with vector FUs at fixed-point32 (**full32**). The accuracy loss due to the lower precision is minimal (i.e., 99.995% - 99.88% ≈ 0.12%). This motivates a solution like VecPAC, which dynamically trades off accuracy with the size of the vector units. VecPAC employs vector FUs with multiple (typically 4, but the approach is scalable) fixed-point16 lanes to maximize area-efficiency. Furthermore, multiple VecPAC vector lanes can be fused together to support the higher precision requirement depending on the application. The trade-off between variations of data formats (e.g., Bfloat16, FP8 [19], MSFP [7], TensorFloat32, or customized data formats) and the vectorization, as well as the architectural implications of CGRAs, will be explored as future work.

IV. VecPAC Architecture

VecPAC is loosely coupled with a central processing unit (CPU) and can be invoked via the accelerator command interface. A direct memory access (DMA) unit loads the data and control signals into the CGRA's data scratchpad memory (SPM) and into the CGRA's control memory, respectively. Figure 4 shows a 4×4 tiles VecPAC design. A tile contains: a set of registers that hold input data either from the data memory or from the other tiles, a configuration memory containing the control signals that indicate the computation to execute and the data routing information for each cycle, a set of FUs that perform the computation, and a crossbar that routes data from or to other tiles (or itself). The left-most tiles are equipped with load and store units to access the data memory. A scalar tile consists of floating-point 32-bit FUs. All operations supported by a scalar tile generate one output per cycle. The floating-point FUs in one tile do not need to interact with the fixed-point FUs on the other tiles as we don't allow hybrid data formats in each data-flow of the target kernel.

A. Hybrid and Interleaved Design

In VecPAC, the scalar and vector tiles are placed on the CGRA fabric in an interleaved fashion and interconnected via a king-mesh network. The interleaved placement allows reaching tiles of the same type in one hop. This facilitates the kernel mapping, since operations of the same type usually are clustered together (e.g., the yellow nodes representing vectorized operations of Figure 1(c)).

B. Vectorization Support

Besides integrating vector FUs, the vector tiles also have wider interconnection channels than the scalar ones. Figure 5(a) and Figure 5(d) show how vector FUs (i.e., vector adder and vector multiplier) support vectorized/SIMD operations. In our 4×4 VecPAC design, each vector FU has 4 lanes. Each lane supports 16-bit fixed-point computation. For example, a vector adder consists of 3 full adders and 1 half adder to enable four simultaneous 16-bit fixed-point additions. Then, the four outputs are packed together and routed to the other tiles. All the operations (including the vectorized ones) recognized in LLVM [15] intermediate representation (IR) have corresponding hardware support (i.e., corresponding FUs). Reduction operations execute in 2 cycles.

C. Precision-Awareness

VecPAC can configure vector FUs to perform operations at different precision by combining lanes together. The vector adder supports: four 16-bit fixed-point additions, two 32-bit

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

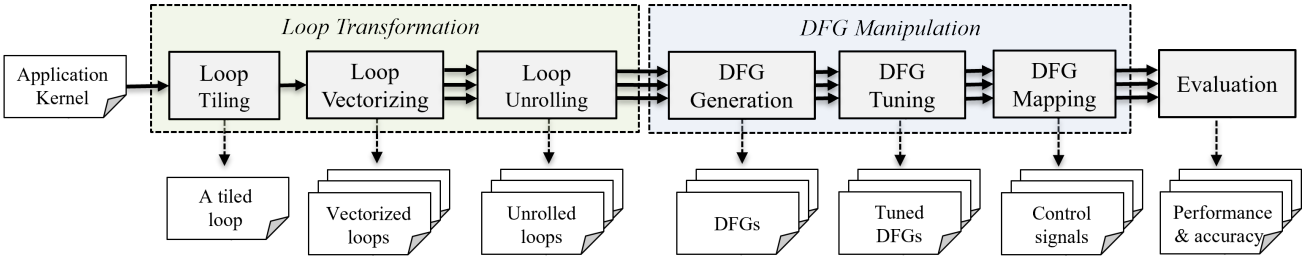


Fig. 6: VecPAC compiler toolchain – The framework is implemented in the LLVM infrastructure [15].

Domain	Workload	Data Size	Unrolling = 1					Unrolling = 4					VecPAC Achieved Accuracy
			Vectorize = 1		Vectorize = 4			Vectorize = 1		Vectorize = 4			
			Opts	Edges	VOpts	Opts	Edges	Opts	Edges	VOpts	Opts	Edges	
Embedded domain	fir	64	11	15	7	13	17	32	42	25	40	50	average 98.1%
	latnrm	32	22	33	-	-	-	70	102	-	-	-	
	fft	1024	32	50	30	42	60	124	190	120	164	230	
	dtw	128^2	27	44	17	32	49	99	164	68	119	184	
Machine learning	spmv	512	24	32	-	-	-	112	149	-	-	-	average 99.627%
	conv	32^2	17	25	8	20	28	51	71	27	60	80	
	relu	1024	16	21	6	16	21	49	66	24	55	72	
High-perf. computing	histogram	2048	15	17	-	-	-	51	56	-	-	-	average 98.31%
	mvt	128^2	20	29	22	42	57	71	104	88	159	216	
	gemm	128^2	12	19	10	17	24	44	66	40	64	86	

TABLE II: Target workloads in three domains – We also apply unrolling by 8 and 16 times for the kernels to map them onto larger (e.g., 8×8) CGRAs. The Opts indicates the number of operations in the corresponding kernel DFG. The VOpts indicates the vectorized operations. `latnrm`, `spmv`, and `histogram` kernels do not have the statistics for vectorization as they are not able to be vectorized using LLVM Auto-Vectorization. The last column presents the average accuracy achieved by VecPAC as shown in Figure 9.

fixed-point additions, and one 64-bit addition by enabling the carry-bit delivery between appropriate sub-adders. A 32-bit fixed-point multiplication, instead, requires four 16-bit multiplications with appropriate shift operations. Therefore, the vector multiplier supports: four parallel 16-bit fixed-point multiplications (as shown in Figure 5(d)) and one 32-bit fixed-point multiplication (shown in Figure 5(e)). If a kernel requires high accuracy, only allowing multiplication at 64-bit fixed point, our approach transforms it into a pattern composed of 5 operations ((1 multiplication, 1 multiplication-shift, and 1 addition) and maps it onto one or more vector tiles. Different mappings can lead to different execution times. Figure 5(f) shows that the output is generated in 2 cycles (without considering input delivery). If all five operations are mapped on the same vector tile, they will take 5 cycles to complete. Usually, as shown in Figure 1(c), vectorized operations are not on the critical path. Hence, multi-cycle execution does not reduce performance, as the CGRA can exploit loop pipelining.

In VecPAC, applications that tolerate accuracy loss can use vector tiles in multi-lane mode, exploiting higher data parallelism and achieving higher speedups. Applications that need high accuracy, instead, employ VecPAC's vector tiles fusing lanes together and using number formats with longer bit-width.

V. VecPAC Compiler Toolchain

We implement the VecPAC compilation framework to facilitate the kernel mapping on VecPAC architecture. An application kernel is described as a DFG, where nodes represent operations and edges indicate data dependencies. The framework maps the DFG on the architecture and is composed of two parts: **Loop Transformation** and **DFG Manipulation**.

A. Loop Transformation

The framework applies loop-level optimizations to a kernel to maximize instruction and data parallelism, considering the architectural constraints.

Loop Vectorization – Loop vectorization transforms the body of a loop so that instead of performing multiple scalar operations of the same type on different elements of a vector, a single operation is simultaneously performed on several vector elements. As previously discussed loop vectorization can improve performance with limited changes to the DFG (see Figure 1(c)). In VecPAC, the vectorization factor is also determined by considering the target accuracy of the application. With the proposed vector FUs, using more lanes means using smaller bit widths, thus less precise number formats that might reduce accuracy. We leverage the LLVM Auto-Vectorization pass [15] to vectorize loops. Not all the kernels are vectorizable due to loop-carried dependency and unsafe dependent memory operations.

Loop Unrolling – A loop body that only has few operations (i.e., with a small DFG) might lead to a mapping that under-utilizes the CGRA's computing resources, hindering performance. Unrolling the loop increases the number of operations (i.e., enlarges the DFG as shown in Figure 1), potentially increasing ILP (if dependencies allow) and improving CGRA's utilization and overall kernel performance.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

B. DFG Manipulation

After applying the loop transformations, the VecPAC compiler generates the DFG, tunes it, and maps it onto the CGRA.

DFG Generation – Each DFG node represents one basic operation, corresponding to one instruction in LLVM supported by a specific FU. Edges represent data dependencies among operations. Control-flow instructions (including branch and phi instructions) are converted to data-flow using partial predication [11]. The vectorized reduce function call in LLVM IR is also represented by a single node of the DFG.

DFG Tuning – If a multiplication can only be performed at 64-bit fixed point precision, the tuning pass splits it into the pattern previously described and shown in Figure 5(f).

DFG Mapping – Our framework implements a heuristic optimization algorithm that maps the DFG onto the CGRA's Modulo Routing Resource Graph (MRRG) [18] targeting the minimum II as the objective. The MRRG is a time-space representation of the CGRA architecture. The algorithm incrementally increases the II until it finds a valid mapping between the DFG and the available hardware resources. The mapping process applies Dijkstra's algorithm to find the shortest path between tiles and perform routing of data communication between operations. The algorithm can only map vector operations on tiles with vector FUs and give higher priority to tiles with scalar FUs for scalar operations. The heuristic algorithm typically takes a few seconds to generate a feasible mapping for a 4×4 CGRA. With larger CGRAs (e.g., 8×8) and larger DFGs (e.g., more than one hundred operations), the algorithm may take several minutes (but generally less than an hour) to find an efficient mapping.

Once the compilation is done, multiple mappings for each target application kernel are generated. Each mapping has a specific speedup with the corresponding accuracy. Users then can pick the mapping of the highest speedup within the accuracy threshold.

VI. Experimental Evaluation

We evaluate VecPAC with applications coming from three different domains: embedded, ML, and HPC. Table II details the characteristics of each evaluated kernel. For the embedded domain, we use digital signal processing (DSP) kernels, i.e., finite impulse response filter (`fir`), normalized lattice filter (`latnrm`), fast-fourier transform (`fft`), and dynamic time warping (`dtw`). The quality of the input data for DSP kernels depends on the analog-to-digital converters, allowing tolerate some losses in accuracy. For the ML domain, we use: sparse matrix-vector multiplication (`spmv`), convolution (`conv`), and rectified linear unit (`relu`). With these kernels, we aim to speed up the execution latency, with minimal impact on the accuracy. To represent the HPC domain, we chose: Histogram (`histogram`), matrix-vector product and transpose (`mvt`), and generalized matrix multiplication (`gemm`). HPC applications require high result accuracy. All the kernels are unrolled with the appropriate unrolling factor to generate large DFGs for larger CGRAs. LLVM Auto-Vectorization cannot vectorize kernels with loop-carried dependency (i.e., `latnrm`) and unsafe dependent memory operations (i.e., `spmv` and `histogram`).

Functional Unit	Area (μm^2)	Tile & CGRA	Area (μm^2)
fixed16 Adder	830	scalar tile 32b	30374
fixed32 Adder	1490	vector tile 16b	48400
float32 Adder	5502	vector tile 32b	72405
fixed16 Multiplier	1350	4x4 CGRA	485984
fixed32 Multiplier	3991	2x2 full32	289620
float32 Multiplier	6467	2x2 VecPAC	157549

TABLE III: Post-synthesis area breakdown of basic arithmetic functional units, tiles, and CGRAs – A vector functional unit in the vector tile contains 4 lanes. Logic synthesis is performed with FreePDK 45nm library, targeting an 800 MHz clock frequency.

We model VecPAC and the other CGRA configurations with PyMTL3 [12], which will generate the synthesizable Verilog code. We then use Yosys open-source tool within mflowgen [1] to perform logic synthesis and obtain the power, area, and timing estimates of the design. For VecPAC's SRAM design, we use CACTI 6.5 [28] to characterize the scratchpad data memory (SPM) performance and VecPAC will perform tiling when data cannot fit. Table III shows the area breakdown of different VecPAC's components on a FreePDK 45nm cell library.

Baseline Design – We compare VecPAC with two baseline CGRA designs: (1) a conventional 4×4 scalar CGRA without precision-awareness [26]; and (2) a CGRA that integrates tiles with **four-lane vector FUs** (each lane supports fixed-point 32-bit precision, denoted as **full32**). Our VecPAC, is a hybrid CGRA design that includes both tiles with scalar FUs and tiles with vector FUs. Vector FUs have 4 lanes each. Each lane supports fixed-point16 but can be fused together to achieve higher data precision. We show several different CGRA configurations of VecPAC throughout the evaluation.

Accuracy Measurement – In our evaluation, different CGRAs might support different number formats. Accuracy is also input dependent: if data fit within a limited range, the fixed-point format provides more precise results than a floating-point format with the same bit width. We generate random input data to alleviate the impact of these phenomena and maintain fairness in the evaluation. For each kernel, we calculate accuracy as the deviation of the output values from the values obtained by performing all computations of the kernel on a CPU using the IEEE754 floating-point 32-bit format.

A. Experimental Results

In this section, we evaluate VecPAC performance, area-efficiency, and scalability. We also discuss how VecPAC can trade off accuracy for performance.

Performance and Area-Efficiency – Figure 7 shows area-efficiency and performance of different CGRA configurations, normalized over the baseline. On average, VecPAC obtains a speed up of 1.1× over the baseline, compared to the 1.15× speed up of **full32**. However, VecPAC integrates fewer vector tiles. CGRAs with vector FUs cannot improve the performance of the `latnrm`, `spmv`, and `histogram` kernels, as these

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

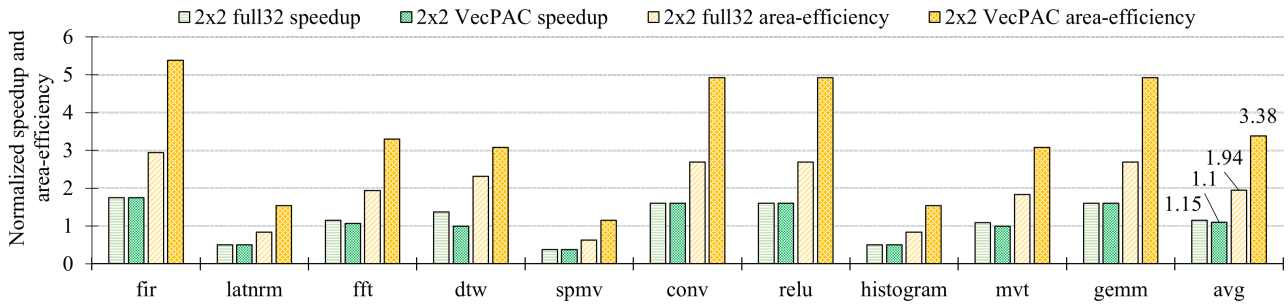


Fig. 7: Normalized performance and area-efficiency w.r.t. the 4×4 scalar CGRA baseline – full32 indicates the vector CGRA without precision-awareness (i.e., each vector lane supports fixed-point32 number format). VecPAC is the hybrid CGRA design containing both vector and scalar tiles. VecPAC achieves a performance similar to the vector CGRA (i.e., 1.1× vs. 1.15×), but significantly improves area-efficiency by 1.74×.

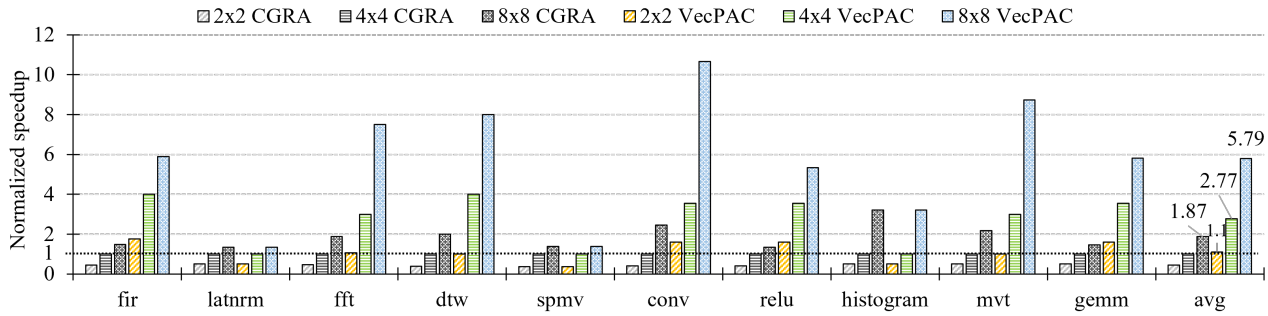


Fig. 8: Scalability of VecPAC – On average, VecPAC outperforms a CGRA with only scalar functional units able to perform a similar amount of parallel operations (e.g., a 4×4 VecPAC has 1.48× the performance of an 8×8 scalar CGRA). For kernels that cannot be vectorized, VecPAC achieves a performance similar to a CGRA with only scalar functional units that have the same number of tiles.

cannot be vectorized. VecPAC significantly improves area efficiency with respect to the baseline (3.38×) and to **full32** (1.74×). Table III breaks down the area of various basic components of the CGRAs. VecPAC occupies 54.4% of the area of a CGRA that has the same number of tiles with vector FUs, but without support for lane fusion and variable precision.

Scalability – Figure 8 shows the performance of VecPAC and of the other CGRAs configurations varying the number of tiles over the baseline. For kernels that can be vectorized, VecPAC's performance scales better as the number of tiles increases than other designs. With larger designs, scalar CGRAs have to mainly leverage loop unrolling to increase ILP. VecPAC can also exploit data parallelism through vectorization (e.g., a 2×2 VecPAC can execute the same number of operations in parallel as a 4×4 scalar CGRA). An 8×8 conventional CGRA is only 1.82× faster on average than the baseline, much lower than the increase in computational resources. A VecPAC design that performs the same number of operations as a scalar 8×8 design, instead, provides a speedup of 2.77× over the baseline. The `relu` kernel does not scale as well as the others on VecPAC, because it has fewer arithmetic operations per memory access. Only tiles attached to the SPM (on the left-most column) can perform memory operations, and on VecPAC only half of these can perform vectorized memory accesses, limiting the performance of kernels that are memory bound.

Trade-off between Performance and Accuracy – Figure 9 shows the trade-off between accuracy and speedup (normalized to the baseline) across the kernels on a 4×4 VecPAC design. Vectorization factors of 1 and 2, indicate that the four vector lanes are fused into one to support 64-bit fixed-point, and into two to support 32-bit fixed-point, respectively. The performance does not change on the kernels that cannot be vectorized, since the LLVM Auto-Vectorization cannot be applied. Average speed up counter-intuitively decreases more than 50% with a vectorization factor of 2 (for example, on `conv`). The reason is that multiplication on fused lanes requires more resources than addition. However, the additional overhead for fusing lanes for multiplication is low, since performance on VecPAC is usually constrained by critical paths consisting of scalar operations. The last column of Figure 9 shows accuracy and normalized speedup when all vector lanes of the VecPAC design execute independently. VecPAC achieves an accuracy of 98.1%, 99.62%, and 98.31% for the embedded, ML, and HPC domains, respectively. For applications that cannot tolerate such accuracy losses, VecPAC can employ the fully fused mode for vector FUs, or tiles with scalar floating point FUs.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

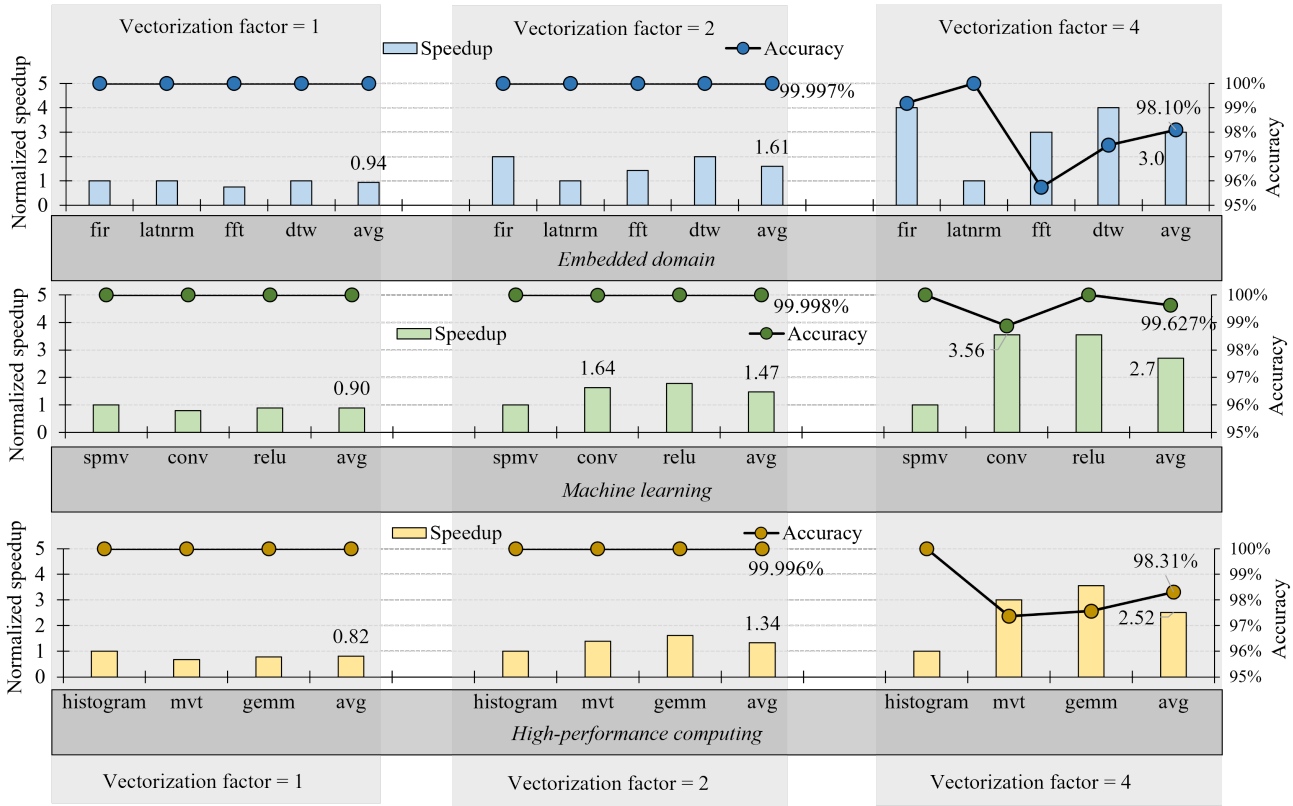


Fig. 9: Trade-off between the normalized performance and the accuracy across different application domains – The speedup is normalized with respect to the execution time on the 4×4 scalar CGRA baseline. The three columns indicate different execution modes of VecPAC.

B. Physical Design with Advanced Technology Node

To verify VecPAC design with a modern advanced technology node, we also implemented an ASIC flow that generates GDSII from VecPAC's RTL with ASAP 7nm FinFET cell library [6]. Figure 10 shows the placed-and-routed 4x4 VecPAC CGRA which occupies 0.169 mm^2 while consuming an average power of 9.65 mW (@ 0.7V/555MHz). Each tile of the CGRA is placed according to the floorplan shown in Figure 4 (b). The shaded area in hunter green in Figure 10 highlights the core logic (i.e., standard cells) inside each CGRA tile.

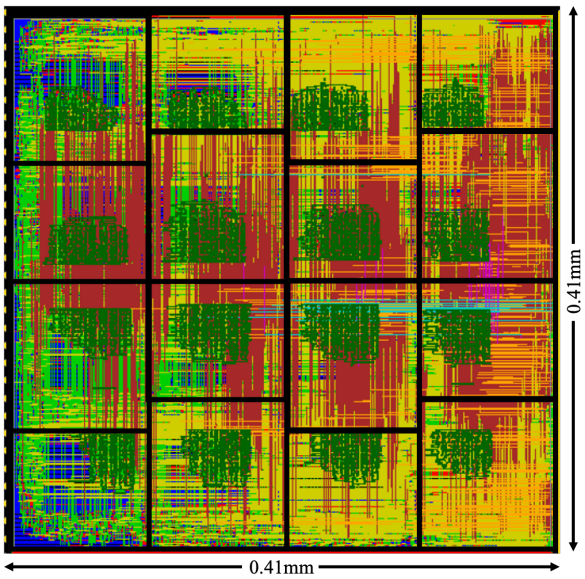


Fig. 10: VecPAC Physical Layout with ASAP 7nm Library.

VII. Conclusion

This paper proposes VecPAC, a hybrid CGRA design that combines tiles with scalar FUs and tiles with configurable vector FUs. The vector functional units provide the capability to fuse together lanes trading off parallelism for accuracy. VecPAC outperforms conventional scalar CGRAs with a similar level of operation parallelism by 1.48×. VecPAC is also more area efficient than comparable CGRAs with tiles featuring only scalar FUs (3.38×) or vector FUs (1.74×).

Acknowledgments

This research was partially supported by the Data Model Convergence (DMC) Initiative, the Adaptive Tunability for Synthesis and Control via Autonomous Learning on Edge (AT SCALE) Initiative, and the Compiler Frameworks and Hardware Generators to Support Innovative US Government (IUSG) Designs project at Pacific Northwest National Laboratory (PNNL). PNNL is a multi-program national laboratory operated for the U.S. Department of Energy (DOE) by Battelle Memorial Institute under Contract No. DE-AC05-76RL01830.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.

References

[1] https://github.com/cornell-brg/mflowgen, Mflowgen.

[2] https://sambanova.ai, SambaNova Systems.

[3] https://cloud.google.com/tpu/docs/bfloat16, Using bfloat16 with tensorflow models.

[4] Omid Akbari, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, and Muhammad Shafique. Px-cgra: Polymorphic approximate coarse-grained reconfigurable architecture. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 413–418. IEEE, 2018.

[5] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro*, 41(2):29–35, 2021.

[6] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. Asap7: A 7-nm finfet predictive process design kit. *Microelectronics Journal*, 53:105–115, 2016.

[7] Bita Darvish Rouhani, Daniel Lo, Ritchie Zhao, Ming Liu, Jeremy Fowers, Kalin Ovtcharov, Anna Vinogradsky, Sarah Massengill, Lita Yang, Ray Bittner, et al. Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point. *Advances in neural information processing systems*, 33:10271–10281, 2020.

[8] Loris Duch, Soumya Basu, Rubén Braojos, Giovanni Ansaloni, Laura Pozzi, and David Atienza. Heal-wear: An ultra-low power heterogeneous system for bio-signal analysis. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 64(9):2448–2461, 2017.

[9] Zahra Ebrahimi and Akash Kumar. Biocare: An energy-efficient cgra for bio-signal processing at the edge. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2021.

[10] Brian Gaide, Dinesh Gaitonde, Chirag Ravishankar, and Trevor Bauer. Xilinx adaptive compute acceleration platform: Versaltm architecture. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 84–93, 2019.

[11] Mahdi Hamzeh, Aviral Shrivastava, and Sarma Vrudhula. Branch-aware loop mapping on cgras. In *Proceedings of the 51st Annual Design Automation Conference*, pages 1–6, 2014.

[12] Shunning Jiang, Peitian Pan, Yanghui Ou, and Christopher Batten. Pymtl3: a python framework for open-source hardware modeling, generation, simulation, and verification. *IEEE Micro*, 40(4):58–66, 2020.

[13] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. Hycube: A cgra with reconfigurable single-cycle multi-hop interconnect. In *Proceedings of the 54th Annual Design Automation Conference 2017*, pages 1–6, 2017.

[14] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices*, 53(2):461–475, 2018.

[15] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.

[16] Leibo Liu, Jianfeng Zhu, Zhaoshi Li, Yanan Lu, Yangdong Deng, Jie Han, Shouyi Yin, and Shaojun Wei. A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications. *ACM Computing Surveys (CSUR)*, 52(6):1–39, 2019.

[17] Stefan Mach, Fabian Schuiki, Florian Zaruba, and Luca Benini. A 0.80 pj/flop, 1.24 tflop/sw 8-to-64 bit transprecision floating-point unit for a 64 bit risc-v processor in 22nm fd-soi. In *2019 IFIP/IEEE 27th International Conference on Very Large Scale Integration (VLSI-SoC)*, pages 95–98. IEEE, 2019.

[18] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Hugo De Man, and Rudy Lauwereins. Exploiting loop-level parallelism on coarse-grained reconfigurable architectures using modulo scheduling. *IEE Proceedings-Computers and Digital Techniques*, 150(5):255, 2003.

[19] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[20] Artur Podobas, Kentaro Sano, and Satoshi Matsuoka. A template-based framework for exploring coarse-grained reconfigurable architectures. In *2020 IEEE 31st International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pages 1–8. IEEE, 2020.

[21] Raghu Prabhakar, Yaqi Zhang, David Koeplinger, Matt Feldman, Tian Zhao, Stefan Hadjis, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. Plasticine: A reconfigurable architecture for parallel patterns. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 389–402. IEEE, 2017.

[22] Rohit Prasad, Satyajit Das, Kevin JM Martin, Giuseppe Tagliavini, Philippe Coussy, Luca Benini, and Davide Rossi. Transpire: An energy-efficient transprecision floating-point programmable architecture. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1067–1072. IEEE, 2020.

[23] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, and Hadi Esmaeilzadeh. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 764–775. IEEE, 2018.

[24] Cheng Tan, Nicolas Bohm Agostini, Jeff Zhang, Marco Minutoli, Vito Giovanni Castellana, Chenhao Xie, Tong Geng, Ang Li, Kevin Barker, and Antonino Tumeo. Opencgra: Democratizing coarse-grained reconfigurable arrays. In *2021 IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pages 149–155. IEEE, 2021.

[25] Cheng Tan, Thierry Tambe, Jeff Zhang, Bo Fang, Tong Geng, Gu-Yeon Wei, David Brooks, Antonino Tumeo, Ganesh Gopalakrishnan, and Ang Li. Asap: automatic synthesis of area-efficient and precision-aware cgras. In *Proceedings of the 36th ACM International Conference on Supercomputing*, pages 1–13, 2022.

[26] Cheng Tan, Chenhao Xie, Ang Li, Kevin J Barker, and Antonino Tumeo. Opencgra: An open-source unified framework for modeling, testing, and evaluating cgras. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*, pages 381–388. IEEE, 2020.

[27] Cheng Tan, Chenhao Xie, Ang Li, Kevin J Barker, and Antonino Tumeo. Aurora: Automated refinement of coarse-grained reconfigurable accelerators. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1388–1393. IEEE, 2021.

[28] Shyamkumar Thoziyoor, N Muralimanohar, J Ahn, and N Jouppi. Cacti 6.5. *hpl. hp. com*, 2009.

[29] Jian Weng, Sihao Liu, Vidushi Dadu, Zhengrong Wang, Preyas Shah, and Tony Nowatzki. Dsagen: Synthesizing programmable spatial accelerators. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 268–281. IEEE, 2020.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 08:17:50 UTC from IEEE Xplore. Restrictions apply.